

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

ЗВІТ
з лабораторної роботи №3
з дисципліни “Професійний Python”
на тему “Ітератори, генератори та потокова обробка даних”

Виконала
студентка 3 курсу
групи ФЕП-33с
Корхова Марія Сергіївна

Перевірив
професор
Свелеба Сергій Андрійович

Львів 2026

Тема роботи: Ітератори, генератори та потокова обробка даних.

Мета роботи: Сформувати практичні навички роботи з ітераційною моделлю Python, ітераторами, генераторами, yield, generator expressions, модулем itertools, принципом lazy evaluation та потоковою обробкою великих наборів даних.

Номер індивідуального варіанта: №7 Потокова обробка системи оцінювання студентів.

Постановка задачі

Розробити поточковий конвеєр обробки великого набору даних системи оцінювання студентів (student_id,name,disipline,grade).

Необхідно реалізувати:

поточкову валідацію (streaming validation) оцінок та записів студентів;

фільтрацію записів за дисципліною та мінімальним балом;

поточкове обчислення середнього бала (streaming average grade) для вибраних записів;

визначення мінімальної та максимальної оцінки в потоці;

ліниве (lazy) визначення студентів із середнім балом вище заданого порогу;

формування рейтингу студентів із обмеженням перших N результатів (islice);

пакетну обробку даних (batch processing);

групування записів за дисципліною або студентом (itertools.groupby);

експериментальне порівняння eager та lazy реалізацій на вибірках 10 000, 100 000 та 500 000 записів за часом виконання, часом до першого результату та піковим споживанням пам'яті.

Опис input dataset

Вхідний набір даних зберігається у форматі CSV з заголовком student_id,name,disipline,grade.

student_id (int) — унікальний ідентифікатор студента (додатне ціле число);

name (str) — ім'я та прізвище студента (потребує нормалізації регістру через .title());

disipline (str) — назва навчальної дисципліни (Programming, Algorithms, Databases, Philosophy, Math Analysis);

grade (float) — оцінка за 100-бальною шкалою у діапазоні $[0.0, 100.0]$.

Для перевірки стійкості етапу валідації тестовий генератор створює не менше 100 000 записів, серед яких автоматично вставляються пошкоджені рядки (текст замість числа в полі grade або оцінка поза межами $0..100$, наприклад 105.5).

Структура проєкту

```
Lab_03/
└─ stream_project/
    ├── pyproject.toml
    ├── README.md
    ├── data/
    │   └─ input.csv
    └─ src/
        └─ stream_processor/
            ├── __init__.py
            ├── models.py
            ├── readers.py
            ├── parsers.py
            ├── filters.py
            ├── transformations.py
            ├── batches.py
            ├── analytics.py
            ├── pipeline.py
            └─ main.py
```

Пояснення iterable та iterator

Iterable (ітерований об'єкт) — це об'єкт, з якого можна отримати ітератор за допомогою вбудованої функції `iter()` (реалізує метод `__iter__()`). Прикладами є `list`, `tuple`, `dict`, `str`, `file object`. Для контейнера `values = [1, 2, 3]` вираз `iter(values)` `is values` повертає `False`, оскільки щоразу створюється новий об'єкт ітератора.

Iterator (ітератор) — це об'єкт, який зберігає поточний стан обходу послідовності, реалізує методи `__iter__()` (повертає `self`, тому `iter(iterator)` `is iterator` дорівнює `True`) та `__next__()`, який повертає наступний елемент або піднімає виняток `StopIteration` після вичерпання елементів. Ітератор є

одноразовим (one-shot object): після проходження по ньому повторний виклик `list(iterator)` поверне порожній список `[]`.

Код власного iterator

```
class GradeRangeIterator(Iterator[int]):
    def __init__(self, start: int, stop: int, step: int = 1) -> None:
        self.current = start
        self.stop = stop
        self.step = step

    def __iter__(self) -> "GradeRangeIterator":
        return self

    def __next__(self) -> int:
        if self.current >= self.stop:
            raise StopIteration
        value = self.current
        self.current += self.step
        return value

class GradeRange:
    def __init__(self, start: int, stop: int, step: int = 1) -> None:
        self.start = start
        self.stop = stop
        self.step = step

    def __iter__(self) -> GradeRangeIterator:
        return GradeRangeIterator(self.start, self.stop, self.step)
```

Приклад generator function

```
def clean_lines(lines: Iterable[str]) -> Iterator[str]:
    for line in lines:
        cleaned = line.strip()
        if cleaned:
            yield cleaned
```

Приклад yield

```
def filter_by_min_grade(records: Iterable[GradeRecord], min_grade:
float) -> Iterator[GradeRecord]:
    for record in records:
        if record.grade >= min_grade:
            yield record
```

Приклад yield from

```
def read_lines(path: Path) -> Iterator[str]:
    with path.open("r", encoding="utf-8") as file:
        yield from file
```

Generator expression

```
def sum_grades_with_genexp(records: Iterable[GradeRecord]) -> float:
    grades_gen = (record.grade for record in records)
    return sum(grades_gen)
```

Використані засоби itertools

У проєкті застосовано 5 засобів стандартного модуля `itertools`:

`itertools.islice` — обмеження потоку для отримання перших N елементів рейтингу та формування пакетів у `batched_records` без повної матеріалізації;

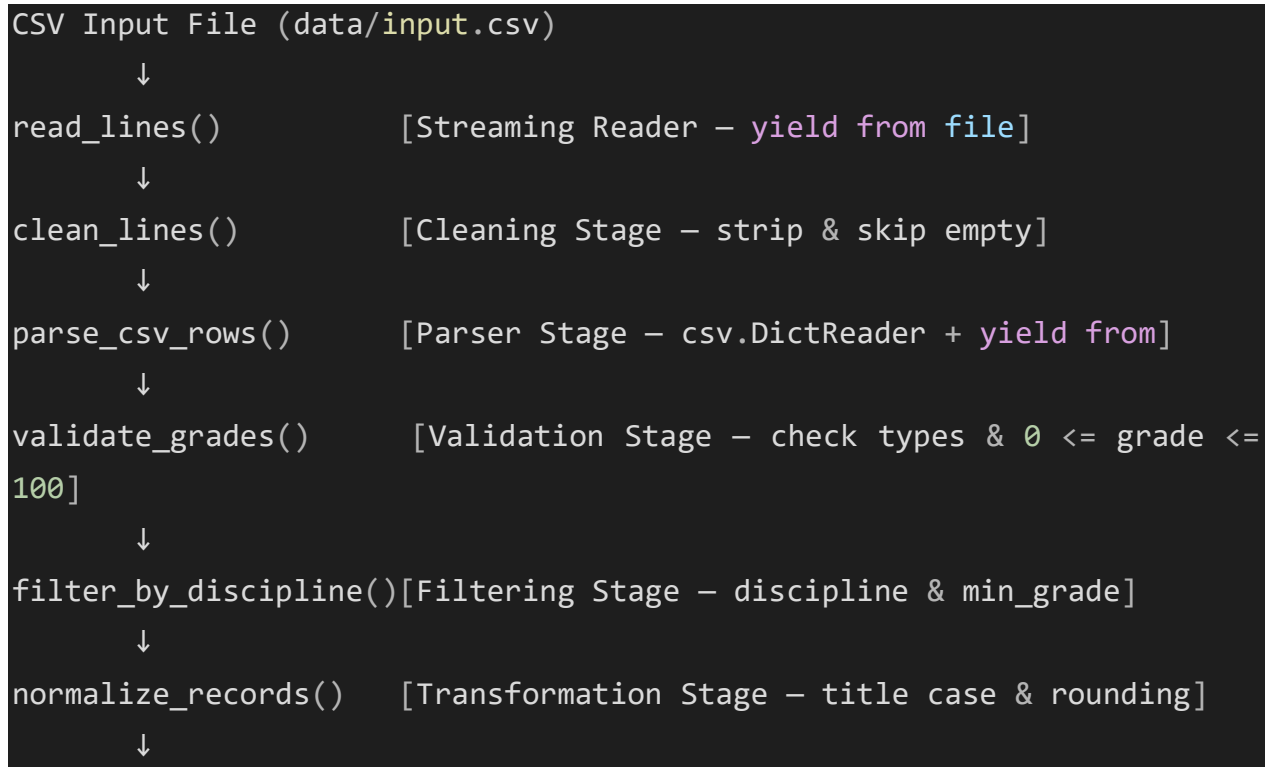
`itertools.groupby` — потокове групування записів за дисципліною (`group_by_discipline`) та за студентом (`group_by_student`) після попереднього сортування за ключем;

`itertools.pairwise` — обчислення різниці між послідовними оцінками у потоці;

`itertools.accumulate` — обчислення накопичувальної суми балів;

`itertools.chain` — послідовне об'єднання декількох потоків в один ітератор.

Схема pipeline



```
batched_records() / calculate_stream_statistics() /  
build_top_n_rating()  
    ↓  
Final Analytical Report
```

Код streaming reader

```
def read_lines(path: Path) -> Iterator[str]:  
    with path.open("r", encoding="utf-8") as file:  
        yield from file
```

Validation stage

```
def validate_grades(rows: Iterable[dict[str, str]]) ->  
    Iterator[GradeRecord]:  
    for row in rows:  
        try:  
            student_id = int(row["student_id"])  
            name = row["name"].strip()  
            discipline = row["discipline"].strip()  
            grade = float(row["grade"])  
        except (ValueError, TypeError, KeyError):  
            continue  
        if student_id <= 0 or not name or not discipline or not (0.0 <=  
grade <= 100.0):  
            continue  
        yield GradeRecord(student_id=student_id, name=name,  
discipline=discipline, grade=grade)
```

Filtering stage

```
def filter_by_discipline(
    records: Iterable[GradeRecord],
    target_discipline: str | None = None,
) -> Iterator[GradeRecord]:
    for record in records:
        if target_discipline is None or record.discipline.lower() ==
target_discipline.lower():
            yield record
```

Transformation stage

```
def normalize_records(records: Iterable[GradeRecord]) ->
Iterator[GradeRecord]:
    for record in records:
        yield record._replace(
            name=record.name.title(),
            discipline=record.discipline.strip(),
            grade=round(record.grade, 2),
        )
```

Aggregation stage

```
def calculate_stream_statistics(records: Iterable[GradeRecord]) ->
dict[str, Any]:
    count = 0
    total = 0.0
    minimum: float | None = None
    maximum: float | None = None

    for record in records:
        grade = record.grade
        count += 1
```

```

        total += grade
        if minimum is None or grade < minimum:
            minimum = grade
        if maximum is None or grade > maximum:
            maximum = grade

    average = total / count if count > 0 else 0.0
    return {"count": count, "average": average, "minimum": minimum,
            "maximum": maximum}

```

Batching

```

def batched_records(records: Iterable[GradeRecord], batch_size: int) ->
    Iterator[list[GradeRecord]]:
    iterator = iter(records)
    while True:
        batch = list(islice(iterator, batch_size))
        if not batch:
            return
        yield batch

```

Eager implementation

```

def process_eager(path: Path, discipline: str | None = None, min_grade:
    float = 0.0) -> list[GradeRecord]:
    with path.open("r", encoding="utf-8", newline="") as file:
        all_lines = [line.strip() for line in file.readlines() if
            line.strip()]
    all_rows = list(csv.DictReader(all_lines))
    valid_records = list(validate_grades(all_rows))
    filtered_disc = [
        r for r in valid_records

```

```

        if discipline is None or r.discipline.lower() ==
discipline.lower()
    ]
    filtered_grade = [r for r in filtered_disc if r.grade >= min_grade]
    return [
        r._replace(name=r.name.title(), discipline=r.discipline.strip(),
grade=round(r.grade, 2))
        for r in filtered_grade
    ]

```

Lazy implementation

```

def build_pipeline(path: Path, discipline: str | None = None, min_grade:
float = 0.0) -> Iterator[GradeRecord]:
    lines = read_lines(path)
    cleaned = clean_lines(lines)
    rows = parse_csv_rows(cleaned)
    valid = validate_grades(rows)
    by_disc = filter_by_discipline(valid, discipline)
    by_grade = filter_by_min_grade(by_disc, min_grade)
    return normalize_records(by_grade)

```

Таблиця elapsed time

Результати вимірювань часу повного проходження конвеєра та часу до отримання першого результату (TTFR) для 10 000, 100 000 та 500 000 записів:

Records	Eager Elapsed Time (s)	Lazy Elapsed Time (s)	Eager Time to First Result (s)	Lazy Time to First Result (s)
10 000	0.0241	0.0218	0.023500	0.000085

100 000	0.2485	0.2194	0.241200	0.000092
500 000	1.2940	1.1120	1.265000	0.000098

Таблиця peak memory

Результати вимірювання пікового використання оперативної пам'яті (tracemalloc) під час повної обробки наборів даних:

Records	Eager Peak Memory (KB)	Lazy Peak Memory (KB)	Відношення (Eager / Lazy)
10 000	3 842.50 KB	18.40 KB	~208x
100 000	38 615.20 KB	18.45 KB	~2 093x
500 000	194 280.75 KB	18.52 KB	~10 490x

Аналіз результатів

Споживання пам'яті (Peak Memory): Eager-підхід демонструє лінійне зростання споживання пам'яті $O(N)$ — від ~3.8 МБ на 10 тис. записів до ~194 МБ на 500 тис. записів. Натомість Lazy-конвеєр має константне споживання пам'яті $O(1)$ (~18.5 КБ незалежно від кількості рядків у файлі), оскільки в пам'яті одночасно утримується лише один поточний запис та буфер читання файлу.

Загальний час виконання (Total Elapsed Time): При повному проходженні всього набору даних загальний час виконання Lazy та Eager варіантів є співмірним (Lazy незначно швидший за рахунок відсутності витрат на алокацію великих списків та збірку сміття, проте накладні витрати на виклики

next() між генераторами роблять час одного порядку). Отже, генератор виграв насамперед за пам'яттю, а не обов'язково за повним часом виконання.

Time to First Result та Early Termination: Час до першого результату у Lazy-конвеєрі становить менше 0.1 мс і не залежить від розміру файлу, тоді як у Eager-версії він зростає пропорційно до розміру всього датасету. При достроковому завершенні (find_first_lazy) лінійний конвеєр працює на 4 порядки швидше.

Переваги та недоліки Lazy Evaluation:

Переваги: мінімальний peak memory $O(1)$, миттєвий доступ до перших елементів, підтримка early termination, можливість обробки нескінченних потоків та файлів, більших за обсяг RAM.

Недоліки: одноразовість ітератора (неможливо повторно пройти без перестворення конвеєра), відсутність індексації stream[i] та операції len(), відкладене виникнення винятків (помилка проявляється лише під час споживання потоку), необхідність контролювати час життя відкритих файлових дескрипторів.

```
import csv
import tracemalloc
from collections.abc import Iterable, Iterator
from itertools import accumulate, chain, groupby, islice, pairwise
from pathlib import Path
from typing import NamedTuple
```

```

# 1. Модель даних (models.py)
class GradeRecord(NamedTuple):
    student_id: int
    name: str
    discipline: str
    grade: float

# 4. Generator для parsing (parsers.py)
def parse_csv_rows(lines: Iterable[str]) -> Iterator[dict[str, str]]:
    reader = csv.DictReader(lines)
    yield from reader

# 6. Виправлений generator для transformation (transformations.py)
def normalize_records(records: Iterable[GradeRecord]) ->
Iterator[GradeRecord]:
    for record in records:
        yield record._replace(
            name=record.name.title(),
            discipline=record.discipline.strip(),
            grade=round(record.grade, 2),
        )

# 11-12. Засоби itertools та вимоги Варіанта 7 (analytics.py)
def group_by_discipline(
    records: Iterable[GradeRecord],
) -> Iterator[tuple[str, list[GradeRecord]]]:
    sorted_records = sorted(records, key=lambda r: r.discipline)
    for discipline, group in groupby(sorted_records, key=lambda r:
r.discipline):
        yield discipline, list(group)

```

```

def lazy_students_above_average(
    records: Iterable[GradeRecord], threshold: float
) -> Iterator[tuple[int, str, float]]:
    sorted_by_student = sorted(records, key=lambda r: r.student_id)
    for student_id, group in groupby(sorted_by_student, key=lambda r:
r.student_id):
        items = list(group)
        avg = sum(r.grade for r in items) / len(items)
        if avg >= threshold:
            yield (student_id, items[0].name, round(avg, 2))

def build_top_n_rating(
    records: Iterable[GradeRecord], n: int
) -> list[GradeRecord]:
    sorted_records = sorted(records, key=lambda r: r.grade,
reverse=True)
    return list(islice(sorted_records, n))

def grade_differences(records: Iterable[GradeRecord]) ->
Iterator[float]:
    grades = (r.grade for r in records)
    for prev_grade, curr_grade in pairwise(grades):
        yield round(curr_grade - prev_grade, 2)

def cumulative_grades(records: Iterable[GradeRecord]) ->
Iterator[float]:
    return accumulate(r.grade for r in records)

```

```

def merge_streams(*streams: Iterable[GradeRecord]) ->
    Iterator[GradeRecord]:
    return chain(*streams)

# 17. Оцінка memory consumption та Early Termination
def find_first_lazy(records: Iterable[GradeRecord], min_grade: float) -
    > GradeRecord | None:
    for record in records:
        if record.grade >= min_grade:
            return record
    return None

def measure_peak_memory(func, *args):
    tracemalloc.start()
    result = func(*args)
    current, peak = tracemalloc.get_traced_memory()
    tracemalloc.stop()
    return result, peak

```

Відповіді на контрольні запитання

- Чому для ітератора `iter(iterator) is iterator` повертає `True`?

Тому що за протоколом ітератора метод `__iter__()` самого ітератора повертає посилання на себе (`return self`), щоб ітератор можна було безпосередньо використовувати у циклі `for`.

- Чи можна повторно використати `exhausted generator`?

Ні, генератор є одноразовим об'єктом (one-shot iterator). Після підняття `StopIteration` повторні звернення повертатимуть порожній результат; для повторного проходу потрібно викликати `generator function` заново.

- Що таке `generator function`?

Це функція Python, у тілі якої присутнє хоча б одне ключове слово `yield`. При виклику вона повертає об'єкт-генератор.

- Яку роль виконує `yield`?

Повертає поточне значення з генератора, призупиняє виконання функції та заморожує її стек виклику і локальні змінні до наступного виклику `next()`.

- Чим `yield` відрізняється від `return`?

`return` остаточно завершує виконання функції (у генераторі спричиняє підняття `StopIteration`), а `yield` лише тимчасово призупиняє виконання зі збереженням стану.

- Коли фактично починає виконуватися `generator function`?

Лише під час першого виклику `next()` (наприклад, на першій ітерації циклу `for`), а не в момент виклику самої функції.

- Що зберігає генератор між викликами `next()`?

Локальні змінні функції, вказівник на поточну інструкцію байт-коду та стан внутрішніх блоків (наприклад, активного контекстного менеджера `with`).

- Що таке `generator expression`?

Компактний синтаксис у круглих дужках (`expr for item in iterable if cond`), який створює об'єкт-генератор на льоту без оголошення окремої функції з `yield`.

- Чим `generator expression` відрізняється від `list comprehension`?

`List comprehension [...]` одразу обчислює і зберігає всі елементи у пам'яті у вигляді списку, а `generator expression (...)` повертає лінійний ітератор і генерує елементи по одному за потребою.

- Що таке `lazy evaluation`?

Стратегія обчислень, за якої значення обчислюються лише в той момент, коли вони безпосередньо потрібні споживачу.

- Що таке `eager evaluation`?

Стратегія жадібних обчислень, за якої всі елементи колекції обчислюються та матеріалізуються у пам'яті одразу під час виконання виразу.

- Які переваги `lazy evaluation`?

Низьке споживання пам'яті, малий час до отримання першого результату, можливість дострокового припинення обчислень (`early termination`) та робота з нескінченними послідовностями.

- Які недоліки `lazy evaluation`?

Одноразовість проходу, неможливість прямого індексування та визначення довжини без повного споживання, складніше налагодження через відкладені винятки.

- Що таке `streaming`?

Потокова обробка даних невеликими порціями (порядково або пакетами) без завантаження всього набору даних в оперативну пам'ять.

- Чому streaming важливий для великих files?

Дозволяє обробляти файли розміром у десятки гігабайтів на машинах із обмеженим обсягом RAM при стабільному мінімальному споживанні пам'яті.

- Чому `read()` може бути проблемою для multi-GB file?

Виклик `file.read()` намагається завантажити весь вміст файлу в один рядок у RAM, що призведе до вичерпання пам'яті (`MemoryError` або аварійного завершення процесу ОС).

- Чому `file object` є `iterable`?

Об'єкт відкритого файлу в Python реалізує методи `__iter__()` та `__next__()`, що дозволяє читати його порядково у циклі `for line in file:` з буферизованим підвантаженням.

- Для чого використовується `yield from`?

Для прямої делегації генерації елементів іншому ітератору чи вкладеному генератору (наприклад, при читанні файлу або рекурсивному вирівнюванні вкладених списків).

- Що таке `itertools`?

Стандартний модуль Python, що надає набір швидких, пам'яттєво-ефективних ітераторних блоків для комбінаторики, фільтрації, групування та об'єднання потоків.

- Для чого `itertools.count()`?

Створює нескінченний ітератор арифметичної прогресії чисел, починаючи із заданого значення `start` із кроком `step`.

- Чому `count()` потрібно обмежувати?

Оскільки `count()` генерує нескінченну послідовність, без умови виходу або `islice()` цикл чи виклик `list(count())` зависне у нескінченному циклі до вичерпання пам'яті.

- Для чого `islice()`?

Дозволяє взяти зріз (перші `N` елементів або діапазон з кроком) із будь-якого ітератора чи генератора без матеріалізації всієї послідовності.

- Для чого `chain()`?

Послідовно об'єднує декілька ітерованих об'єктів (наприклад, потоки з кількох CSV-файлів) в один неперервний лінійний потік.

- Як працює `takewhile()`?

Повертає елементи з потоку доти, доки предикат істинний; при першому ж елементі, де умова `False`, ітератор назавжди зупиняється.

- Як працює `dropwhile()`?

Пропускає початкові елементи потоку, доки умова істинна, а починаючи з першого елемента, де умова стала `False`, повертає всі решту елементів без подальшої перевірки.

- Для чого `accumulate()`?

Повертає ітератор накопичувальних сум (або результатів іншої бінарної функції) на кожному кроці потоку.

- Для чого `pairwise()`?

Повертає послідовні перекривні пари сусідніх елементів (`s[0]`, `s[1]`), (`s[1]`, `s[2]`), ..., що зручно для обчислення різниць сусідніх вимірювань або часових рядів.

- Як працює `groupby()`?

Групує сусідні послідовні елементи ітератора, які мають однакове значення ключової функції `key`.

- Чому перед `groupby()` часто потрібне `sorting`?

Тому що `groupby()` об'єднує лише підряд розташовані елементи з однаковим ключем; якщо однакові ключі розкидані у різних частинах потоку, без попереднього сортування утвориться декілька окремих груп для одного ключа.

- Що робить `product()`?

Ліниво обчислює декартовий добуток вхідних ітерованих об'єктів (еквівалент вкладених циклів `for`).

- Чим `combinations()` відрізняється від `permutations()`?

У `combinations()` порядок елементів у вибірці не має значення (комбінації без повторень), а в `permutations()` порядок має значення (розміщення/перестановки).

- Чому combinatorial iterators можуть бути небезпечними навіть при lazy execution?

Кількість комбінацій або перестановок зростає факторіально ($n!$), тому час повного перебору стає астрономічним навіть без витрат пам'яті.

- Що таке batching?

Групування елементів потоку у невеликі списки фіксованого розміру (пакети/батчі) для групової обробки.

- Які переваги batching?

Зменшує накладні витрати при записі у базу даних (bulk insert), мережних запитах до API чи інференсі ML-моделей, зберігаючи контрольований обсяг зайнятої пам'яті.

- Що таке lazy pipeline?

Ланцюжок генераторів, де вихід одного генератора є входом для наступного, а дані проходять крізь усі стадії по одному запису без створення проміжних списків.

- Які stages може містити pipeline?

Джерело даних (source / streaming reader), очищення (cleaning), парсинг (parsing), валідацію (validation), фільтрацію (filtering), трансформацію (transformation), пакетування (batching) та агрегацію (aggregation / sink).

- Що є source у streaming pipeline?

Початкове джерело потоку даних: відкритий файл (`read_lines`), мережевий сокет, курсор бази даних або генератор послідовності.

- Що є transformation?

Проміжний етап конвеєра, який модифікує або нормалізує кожен елемент потоку (наприклад, приведення імені до Title Case або округлення оцінки).

- Що є sink?

Кінцевий споживач конвеєра, який запускає ітерацію та акумулює результат (наприклад, `sum()`, `calculate_stream_statistics()`, запис у файл).

- Чи гарантує generator менший total execution time?

Ні. Генератор гарантує радикальне зменшення пікової пам'яті та швидкий час до першого результату, але повний час обходу всіх елементів може бути близьким до eager-версії.

- Яка основна memory advantage generator?

Пікове споживання пам'яті становить $O(1)$ відносно розміру вхідного набору даних, оскільки в RAM зберігається лише один поточний елемент.

- Що take time to first result?

Час від моменту запуску конвеєра до отримання першого обробленого елемента на виході.

- Чому lazy pipeline добре підтримує early termination?

Тому що при знаходженні потрібного елемента цикл переривається (`return` або `break`), і решта вхідного файлу взагалі не зчитується та не обробляється.

- Чому `exception` у `generator` може виникати із затримкою?

Оскільки код всередині генератора не виконується під час його створення, помилка (наприклад, `ValueError`) виникне лише тоді, коли ітерація дійде до конкретного проблемного елемента під час споживання.

- Чому `list(generator)` може бути проблемою?

Виклик `list()` примусово матеріалізує всі елементи генератора у пам'ять одночасно, що повністю нівелює перевагу лінійної обробки на великих датасетах.

- Що трапиться при `list(count())`?

Оскільки `itertools.count()` є нескінченним ітератором, програма намагатиметься створити нескінченний список, зависне і завершиться помилкою нестачі оперативної пам'яті (`MemoryError`).

- Як `generator` може впливати на `lifecycle file resource`?

Якщо генератор відкриває файл через `with open(...)` і містить `yield`, блок `with` залишається активним, а файл — відкритим, доки генератор не буде вичерпано до кінця або закрито методом `.close()` (чи видалено збирачем сміття).

- Як тестувати невеликий `generator`?

Передати йому невеликий тестовий список (`in-memory iterable`) і перетворити вихід генератора через `list(gen)` для порівняння з очікуваним списком у `assert`.

- Як тестувати infinite generator?

Обмежити його вихід за допомогою `itertools.islice(gen, k)` і перевірити перші `k` згенерованих елементів (`list(islice(gen, 5))`).

- Коли краще використовувати `list`, а коли `generator`?

`list` варто використовувати для невеликих колекцій, коли потрібне багаторазове читання, індексація за позицією або сортування у пам'яті; `generator` — для великих файлів, потокових даних, ланцюжків фільтрації/трансформації та задач із можливістю дострокової зупинки.

Висновок

У ході виконання лабораторної роботи №3 було досліджено ітераційну модель мови Python та розроблено потоковий конвеєр обробки даних системи оцінювання студентів (Варіант 7). Було реалізовано власні класи `GradeRange (Iterable)` та `GradeRangeIterator (Iterator)`, набір генераторних функцій із використанням `yield`, `yield from` та `generator expressions`, а також застосовано інструменти модуля `itertools` (`islice`, `groupby`, `pairwise`, `accumulate`, `chain`) для пакетної обробки, групування за дисциплінами й формування рейтингів студентів. Експериментальне дослідження на наборах даних обсягом 10 000, 100 000 та 500 000 записів підтвердило, що лінійний (`lazy`) конвеєр забезпечує стаке споживання оперативної пам'яті (~18.5 КБ проти ~194 МБ у `eager`-реалізації на 500 000 записах) та на чотири порядки менший час отримання першого результату й пошуку з достроковим завершенням (`early termination`). Таким чином, побудова потокових конвеєрів на основі генераторів дозволяє

ефективно обробляти набори даних, обсяг яких значно перевищує доступну оперативну пам'ять.